

API Design Cheat Sheet

REST · Versioning · Pagination · Idempotency · Error Envelopes · Async APIs

HTTP Status Codes			HTTP Verbs		
Code	Meaning	Use when	Verb	Action	Idempotent?
200	OK	Successful GET / PUT	GET	Read resource	■ Yes
201	Created	POST created a resource	POST	Create / trigger	■ No*
202	Accepted	Async job queued	PUT	Replace resource	■ Yes
204	No Content	DELETE succeeded, no body	PATCH	Partial update	■ No*
400	Bad Request	Client sent invalid data	DELETE	Remove resource	■ Yes
401	Unauthorized	Missing / bad credentials	* Make POST/PATCH idempotent with an Idempotency-Key header.		
403	Forbidden	Auth ok, but no permission			
404	Not Found	Resource doesn't exist			
409	Conflict	Duplicate / state conflict			
422	Unprocessable	Valid JSON, invalid semantics			
429	Too Many Requests	Rate limit hit			
500	Internal Server Error	Unexpected server failure			
503	Service Unavailable	Downstream dep is down			

Error Envelope		Versioning		
Every error — always the same shape: <pre>{ "error": { "code": "URL_NOT_FOUND", "message": "No URL for code xyz", "request_id": "a3f9..." } }</pre> Always include request_id — it's the only way to correlate logs to a user complaint.		Strategy	Example	Verdict
		URL prefix	/v1/urls	■ Use this
		Header	API-Version: 2024-01	■■ Harder to test
		Content-Type	vnd.v2+json	■ Too complex
		Query param	?version=1	■ Pollutes URLs
Rule	Detail	Breaking vs non-breaking changes:		
Machine-readable code	SCREAMING_SNAKE — for programmatic handling	Non-breaking ■	Breaking ■	
Human message	Readable, no stack traces in prod	Add optional field	Rename / remove field	
request_id	Propagated from X-Request-ID header	Add new endpoint	Change field type	
Never 200 on error	200 {error:...} is a lie — use real status codes	Add enum value	Change status code meaning	
		Looser validation	Remove endpoint	
		Sunset headers: give clients 6+ months before removing a version.		

Pagination

Idempotency Keys

Type	Pros	Cons	Problem: network drops after server writes, before response arrives. Client retries. Two charges.	
Offset <code>?page=N</code>	Jump to any page	Slow at depth — scans all prior rows	POST /v1/shorten Idempotency-Key: 550e8400-e29b-41d4 { "url": "https://example.com" }	
Cursor <code>?after=id</code>	Fast at any depth, stable	No page jumps, no total count	Rule	Why
Keyset <code>id > last</code>	DB-native, indexed	Composite sort is tricky	Client generates the key	Server can't — it needs to respond first
			UUID v4, client-side	Guaranteed unique per attempt
			Store key + result in DB	Return stored result on retry
			Expire after 24h	Don't store forever
			Scope per user + endpoint	Key from user A ≠ key from user B

Cursor response shape:

```
{ "data": [...], "pagination": { "next_cursor": "dXNlcjox", "has_more": true } }
```

Rule: offset is fine under ~10k rows. Beyond that, switch to cursor.

Only needed on POST/PATCH. GET / PUT / DELETE are already safe to retry.

Async API Pattern — 202 Accepted + Polling / Webhooks

Step	Request / Response	Notes
1. Submit	POST /v1/jobs → 202 + {job_id}	Return immediately — never block the client
2. Poll	GET /v1/jobs/:id → {status: pending done failed}	Client polls with backoff: 1s → 2s → 4s...
3. Result	GET /v1/jobs/:id → {status: done, result: {...}}	Keep result for 24h after completion
3b. Webhook	POST {job_id, status, result}	Server pushes — client must ack with 200

	Polling	Webhooks
Client complexity	Simple — just loop	Must expose a public endpoint
Latency	Depends on poll interval	Near real-time
Reliability	Client controls retries	You must retry on client 5xx
Use when	Internal tools, simple CLIs	User-facing events, integrations

Golden Rules

1	Version from day one.	Adding /v1/ later breaks every existing client URL.
2	Consistent error envelope — always.	One shape. Machine-readable code + human message + request_id.
3	Cursor > offset beyond 10k rows.	Offset scans all skipped rows every time. Cursor doesn't.
4	Idempotency keys on all POST/PATCH.	Networks are unreliable. Retries happen. Be ready.
5	202 for anything >100ms.	Don't make clients wait. Queue it, return job_id, let them poll.
6	Never 200 with an error body.	HTTP status codes exist for a reason. Use them.
7	Propagate request_id end to end.	It's the only way to debug a user complaint in production.
8	Document breaking vs non-breaking changes.	Anything that removes or renames a field is breaking. Period.